

1. Product lookup hides API/database failures

This is the code in `services/database/products/getProductById.ts`. The problem is that both of these situations return null. Then `ProductProvider.tsx` receives null and shows: “No products found for that barcode.”

So imagine the internet is disconnected and the user scans a coke, and then the Firestore fails and service returns null, the app says “No products found”. That message is misleading because the product may exist—the application simply could not contact the service.

```
export default async function getProductByBarcode(barcode: string): Promise<Product | null> {
  try {
    const ref = doc(fdb, "PRODUCTS", barcode);
    const snap = await getDoc(ref);

    if (!snap.exists()) {
      return null;
    }

    const productData = snap.data();
    return normaliseFirestoreProduct({
      id: snap.id,
      ...productData,
      barcode,
    }) as Product;
  } catch (err) {
    console.error("[Firestore] getProductByBarcode error:", err);
    return null;
  }
}
```

Recommendation:

The service should distinguish: 404/product does not exist from network unavailable, service unavailable, timeout, permission/database error

For example: Unable to load product information. Check your connection and try again.

2. Log in

This is part of the codes from `services/authentication/signInWithEmail.ts`, the code already handles invalid emails, wrong passwords, user not found, invalid credentials. Which is good, but for example a network error such as “auth/network-request-failed” falls into the section which will return a message “An unexpected error occurred. Please try again later.” The user cannot tell whether their password is wrong, or the firebase is not available, or their internet connection is down.

```

22     if (error.code === 'auth/invalid-email') {
23         return { success: false, message: 'Invalid Email' };
24     } else if (error.code === 'auth/wrong-password') {
25         return { success: false, message: 'Incorrect Password' };
26     } else if (error.code === 'auth/user-not-found') {
27         return { success: false, message: 'No account with that email was found' };
28     } else if (error.code === 'auth/invalid-credential') {
29         // Firebase returns this when the supplied credential is malformed or invalid.
30         // For email/password sign-in, this usually indicates incorrect email or password.
31         return { success: false, message: 'Invalid credentials. Check your email or password.' };
32     } else {
33         console.error('Authentication error:', error);
34         return { success: false, message: 'An unexpected error occurred. Please try again later.' };
35     }
36 }
37 }

```

Recommendation:

Handle network errors separately: Unable to connect. Check your internet connection and try again.

3. Firestore recommendation candidate search

This is part of the codes from services/database/products/getCandidatesForRecommendations.ts

It performs a Firestore query: where("categories", "array-contains-any", specific). There is currently no local try/catch, retry or timeout in that function.

If Firestore fails, the error propagates upward.

```

const productsRef = collection(fdb, "PRODUCTS");
const q = query(
  productsRef,
  where("categories", "array-contains-any", specific),
  limit(Math.max(1, Math.min(maxPool, 300)))
);

```

The higher recommendation provider catches errors and sets: Failed to load recommendations
but no retry occurs.

4. No global network/offline detection is currently being used

The project has: `@react-native-community/netinfo` installed.

This library can tell the application whether the phone has internet connectivity.

However, I searched the source and couldn't find it actually being used.

So currently the application doesn't appear to have a central: Offline, Back online, Waiting to sync state.

Recommendation:

Frontend should use NetInfo to:

- detect offline state;
- avoid unnecessary API requests;
- show an offline banner;
- enable cached/local features;
- retry synchronisation when connectivity returns.

5. 7-day meal plan

The 7-day meal plan route loads products from Firestore; calls `/api/products/classify` for each product and builds a meal plan.

The classify request checks:

```
if (!res.ok) {  
  throw new Error(...)  
}
```

Like this:

```
// ...  
if (!res.ok) {  
  const text = await res.text();  
  throw new Error(`Classification API failed (${res.status}): ${text}`);  
}
```

Then the outer handler catches failures and returns:

Error: "SERVER_ERROR",
message: "..."

Like this:

```
{  
  error: "SERVER_ERROR",  
  message: err?.message ?? "Unexpected error while generating 7-day meal plan.",  
},
```

With HTTP 500.

The main problem is classification is done using: **Promise.all(...)**

That means if one classification call fails, the entire **Promise.all** fails.

So potentially for example 199 products classify successfully and 1 API request fails, then the entire meal-plan request fails.

There is also no timeout or retry in **classifyViaApi**.

Recommendation:

Use:

- timeout
- limited retry for transient failures
- Promise.allSettled()
- skip failed products where safe
- only fail the whole meal plan when there are not enough reliable products